L'Optimisation Structurale et les Algorithmes Génétiques

M. Montemurro ENSAM-Talence

EVSCP - M. Montemurro

Bibliographie

1. D. E. Goldberg. *Genetic algorithms*. New York: Addison and Wesley, 1994

2. Z. Michalewicz. *Genetic algorithms + data structures = evolution programs*. Berlin: Springer, 1994

3. L. J. Fogel, A. J. Owens, and M. J. Walsh. *Artificial intelligence through simulated evolution*. Chichester: John Wiley, 1966

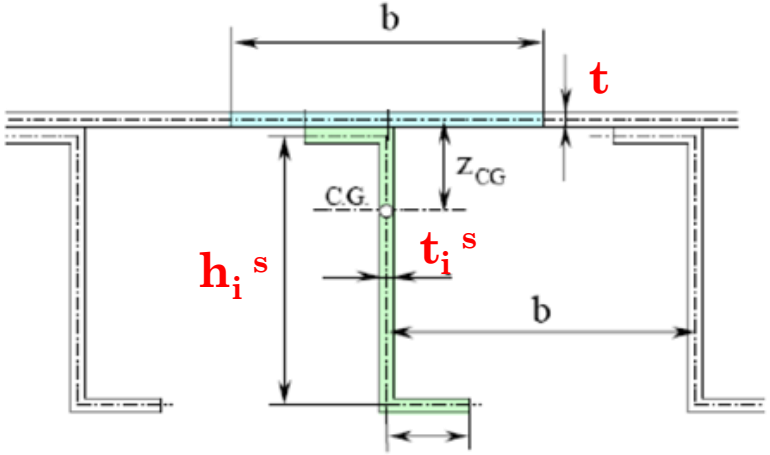
4. F. Glover. Heuristics for integer programming using surrogate constraints. *Decision Sciences*, 8(1):156–166, 1977

5. J. H. Holland. *Adaptation in natural and artificial systems*. Ann Arbor: University of Michigan Press, 1975

6. J. M. Renders. *Algorithmes génétiques et réseaux de neurones*. Paris: Hermes (in French), 1995

Optimisation Structurale

Optimisation Paramétrique Classique



$$d_i^s = 0.3\, h_i^s$$

→ UN PARAMÉTRAGE DU MODÈLE EST NÉCESSAIRE !

Objectif: maximiser (ou minimiser) une propriété particulière de la structure en fonction des variables de conception

Variables de Conception: les paramètres caractéristiques de la structure (dimensions géométriques, propriétés du/des matériau/x, etc.)

Résultat: nouvelle configuration de la structure caractérisée par les paramètres optimaux

Optimisation Structurale

Optimisation Topologique



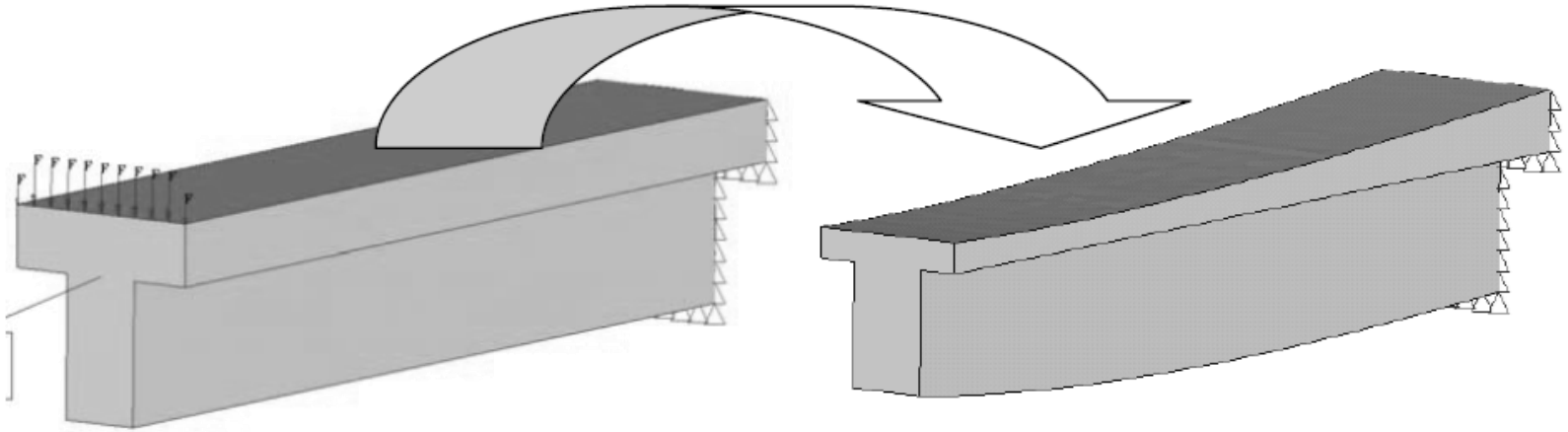
Objectif: maximiser (ou minimiser) une propriété particulière de la structure en fonction des variables de conception

Variables de Conception: les « densités fictives » des éléments composant le modèle (cette densité peut prendre les valeurs entre 0 et 1)

Résultat: distribution optimale de la matière dans un domaine donné (domaine de départ)

Optimisation Structurale

Optimisation de Forme



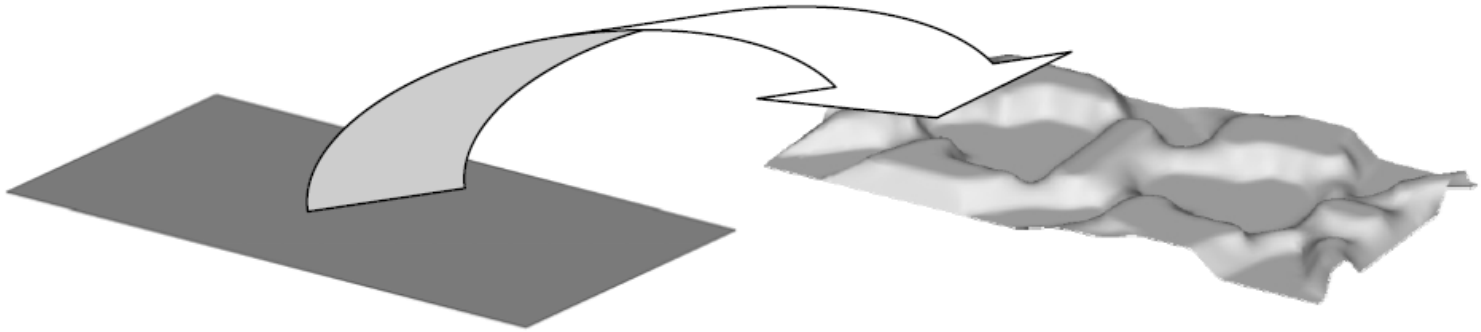
Objectif: maximiser (ou minimiser) une propriété particulière de la structure en fonction des variables de conception

Variables de Conception: coordonnées des « points de contrôle » du contour de la structure

Résultat: forme optimale de la géométrie du modèle

Optimisation Structurale

Optimisation Topographique



C’est une sous-classe de l'optimisation de forme.

Objectif: maximiser (ou minimiser) une propriété particulière de la structure en fonction des variables de conception

Variables de Conception: altimétrie des nœuds des surfaces externes de la structure

Résultat: forme optimale des surfaces composants le contour du modèle (souvent via la création des renforts)

Optimisation Structurale: formulation du problème

Problème de Programmation Non-Linéaire (NLPP)

$$\min_{\mathbf{x}} \Phi(\mathbf{x})$$

$$\begin{cases} g_i(\mathbf{x}) \le 0 & i = 1,\dots,r \\ h_j(\mathbf{x}) = 0 & j = 1,\dots,m \\ \mathbf{x}_L \le \mathbf{x} \le \mathbf{x}_U & \mathbf{x}_L, \mathbf{x}_U, \mathbf{x} \in \Re^n \end{cases}$$

$\Phi(\mathbf{x})$ **Fonction Objectif**: réponse principale de la structure à minimiser ou maximiser

$g_i(\mathbf{x})$ **Contraintes d'Optimisation**: réponses secondaires du modèle qui
$h_j(\mathbf{x})$ doivent satisfaire à certaines conditions

$\mathbf{x}, \mathbf{x}_L, \mathbf{x}_U$ **Vecteur des variables de conception et ses bornes**

Solution Faisable: celle qui satisfait toutes les contraintes du NLPP

Méthodes numériques de résolution du NLPP

Aspects à prendre en compte dans le choix d'une méthode numérique pour la résolution du NLPP

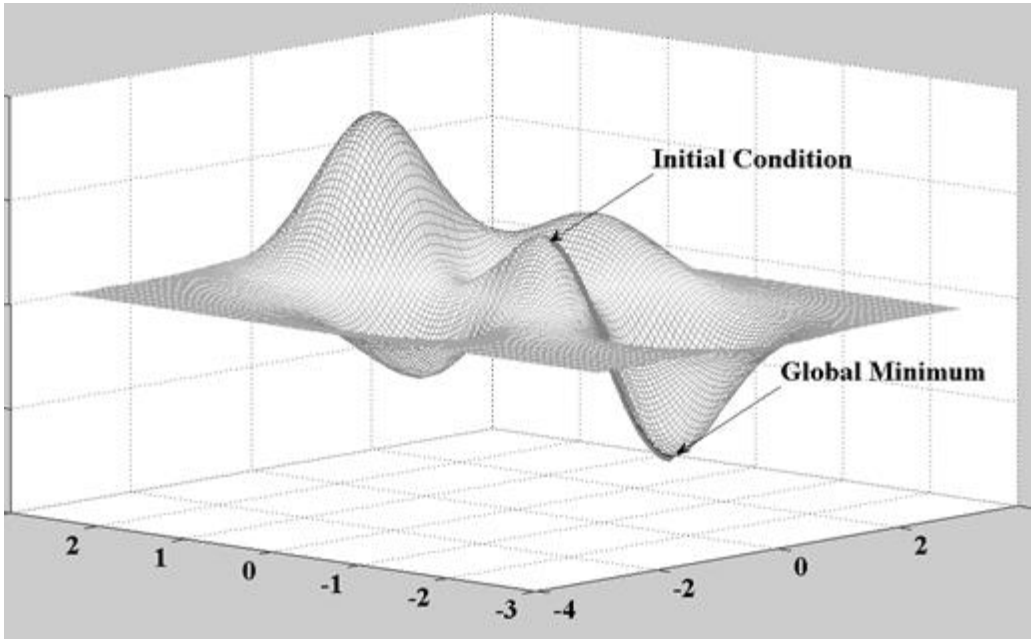
1. Nature des variables (continues, discrètes, abstraites, etc.)
2. Continuité et dérivabilité des fonctions (objectif et contraintes)
3. Présence de contraintes d'optimisation éventuelles
4. Nature des fonctions objectif et contraintes (non-linéarité, non-convexité, etc.)

Méthodes Classiques

Métaheurstiques

Méthodes Classiques

Démarche: on choisit un point de départ faisable et on descende vers un minimum via un ***algorithme de descente***



Défauts:

1. Sensibilité de la solution par rapport au point de départ: pour un NLPP non-convexe on n'a pas la certitude de trouver l'optimum global
2. Traitement des contraintes: il faut empêcher à l'algorithme de poursuivre son chemin au-delà de la frontière → difficulté dans la recherche des optima localisés sur la frontière

Méthodes Classiques

Contexte minimum d'applicabilité:

1. Fonctions continues et dérivables de variables continues
2. NLPP convexes sans contraintes

Contexte élargi d'applicabilité (avec prise en compte de modifications dans la structure des algorithmes):

1. NLPP convexes avec contraintes (méthodes de pénalisation et/ou barrières)
2. NLPP non-convexes (tirage aléatoire des points de départ)
3. NLPP non-convexes avec contraintes

REMARQUE: les méthodes classiques ne peuvent jamais être appliquées aux cas suivants

- NLPP avec variables discrètes ou abstraites
- NLPP avec fonctions objectifs et/ou contraintes discontinues et/ou non-dérivables
- NLPP avec domaines de recherche disjoints

Métaheuristiques

Définition: méthode d'optimisation globale pour NLPP non convexes et qui est guidée dans sa stratégie de recherche par l'usage organisé d'une ou plusieurs règles empiriques, qui s'inspirent d'un quelque phénomène en le reproduisant synthétiquement

CARACTÉRISTIQUES ET AVANTAGES

- **Approches Stochastiques:** elles se basent sur la génération d'une grande quantité de nombres aléatoires et sur des manipulations probabilistes
- **Méthodes d'ordre 0:** elles ne nécessitent que des valeurs de la fonction objectif et des contraintes sans considérer des informations supplémentaires (p. ex. les dérivées)
- Prise en compte de **tout type de variables**: discrètes, continues et abstraites
- Traitement de NLPP de **grand taille** (milliers de variables)
- Elles sont de programmation simple et à schéma relativement libre

Métaheuristiques

CARACTÉRISTIQUES ET AVANTAGES

- **Méthodes d'exploration et d'exploitation:** elles représentent des véritables méthodes d'exploration du domaine de recherche (aussi bien de la région faisable que de celle non-faisable) qui sont capables, en même temps, d'exploiter de façon astucieuse toute l'information pour trouver des optima globaux faisables

- **Auto-organisation:** dans l'exploration du domaine le hasard a un rôle fondamental; cependant cette exploration n'est pas totalement aléatoire mais elle se base sur une « gestion intelligente » de l'information que l'algorithme tire de l'exploration elle-même

- **Intelligence artificielle**: l'auto-organisation intrinsèque des métaheuristiques traite l'information et guide l'algorithme vers les optima globaux en absence d'une autorité centrale

Métaheuristiques

Les différents métaheuristiques

- méthode de Monte-Carlo, inspirée au jeu de hasard
- recuit simulé, inspiré à la thermodynamique
- méthode tabou, inspirée à la gestion de la mémoire
- métaphores biologiques, inspirées au monde du vivant:
  - réseaux de neurones, inspirés au fonctionnement du cerveau
  - réseaux immunitaires, inspirés au fonctionnement du système immunitaire
  - **algorithmes génétiques**, inspirés à la génétique et aux lois de l'évolution
  - colonies de fourmis, inspirées au comportement social des fourmis
  - *particle swarms*, inspirés au comportement dynamique des volées d'oiseaux
  - etc.

Métaphores Biologiques

- **Idée de base des MB:** simuler les mécanismes et les lois propres à la biologie, afin de les utiliser dans des problèmes de nature différente, (souvent utilisés avec des simplifications non plausibles dans la réalité biologique)

- **Les MB s'inspirent des lois de la Nature**: très grande efficacité dans le traitement de l'information distribuée

- Les systèmes biologiques ayant capacité d'adaptation sont des **Réseaux Adaptatifs Non-linéaires** (**RAN**)

- **RAN:** système constitué d'un grand nombre d'unités élémentaires tel que:
  - les unités forment un réseau qui traite l'information de façon parallèle et distribuée
  - les unités interagissent entre elles de façon non linéaire et compétitive sans une autorité centrale
  - des opérateurs modifient les unités pour que le système global (l'ensemble des unités) s'adapte progressivement à l'environnement

Algorithmes Génétiques

Les algorithmes génétiques (**J. Holland**, 1965) sont des **algorithmes d'exploration** fondés sur les mécanismes de la **sélection naturelle** et de la **génétique**.

Nature	AG
Chromosome	Chaine
Gène	Trait
Allèle	Valeur du trait
Locus	Position du trait dans la chaine
Génotype	Structure
Phénotype	Structure décodée
Épistasie	Non-linéarité

Caractéristiques des AG

1. Codage des paramètres
2. Population de points
3. Méthodes d'ordre 0

L. J. Fogel, 1966
I. Rechenberg, 1973
F. Glover, 1977
D. E. Goldberg, 1994
Z. Michalewicz, 1994
M. Schoenauer, 1994

Algorithmes Génétiques

- L'idée à la base des AG est très simple: faire **évoluer** une population en se basant sur la **survie des individus** les mieux adaptés

- En génétique synthétique un **individu** est un **candidat à la solution d'un problème donné**, donc un nombre ou un ensemble de nombres (vecteur, tenseur, matrice etc.)

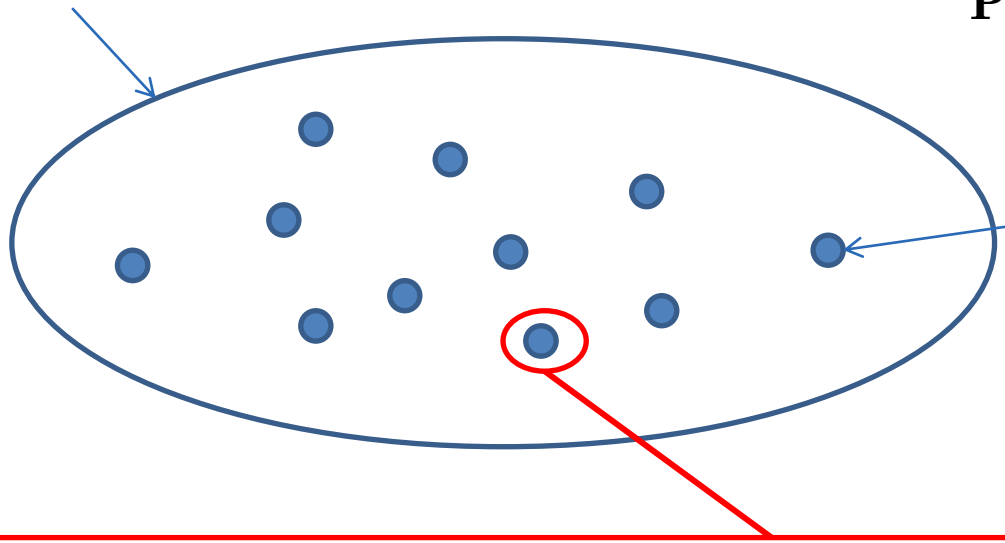
- L'évolution de la population est réalisée en simulant les **mécanismes de transmission** des caractères propres à la génétique, après avoir codé les variables (simulation de l'ADN, Watson e Crick, 1953)

- La différence principale entre les AG et les autres MB est l'introduction du **sexe pour accélérer et améliorer l'adaptation** (la probabilité de s'améliorer pour les êtres vivants est accrue si le patrimoine génétique vient de deux individus plutôt que d'un seul)

Algorithmes Génétiques

Domaine de recherche

Population = *N* Individus



Individu: point solution dans l'espace

Structure de l'individu

gène *1*	gène *2*	gène *3*	…	gène *n*

Chromosome

Gène: variable de conception *codée*

REMARQUE: dans les AG standard l'individu a, généralement, une structure mono-chromosomique

Algorithmes Génétiques

CODAGE BINAIRE DES VARIABLES

$x \in [a, b] \subset \mathbb{R}$ $\qquad n_{var} = 1 + \dfrac{b-a}{p_{var}}$

- x: variable de conception (scalaire)
- a, b: bornes
- p_{var}: finesse de discrétisation
- n_{var}: nombre des valeurs de la variable x

$\boldsymbol{x} = \{x_1, x_2, \dots, x_{n_{var}}\}$ Vecteur des **valeurs** de x

$x_i = a + (i-1)p_{var}$ $\qquad i = 1, \dots, n_{var}$

$\boldsymbol{i} = \{1, \dots, n_{var}\}$ Vecteur des **indices** des valeurs de x

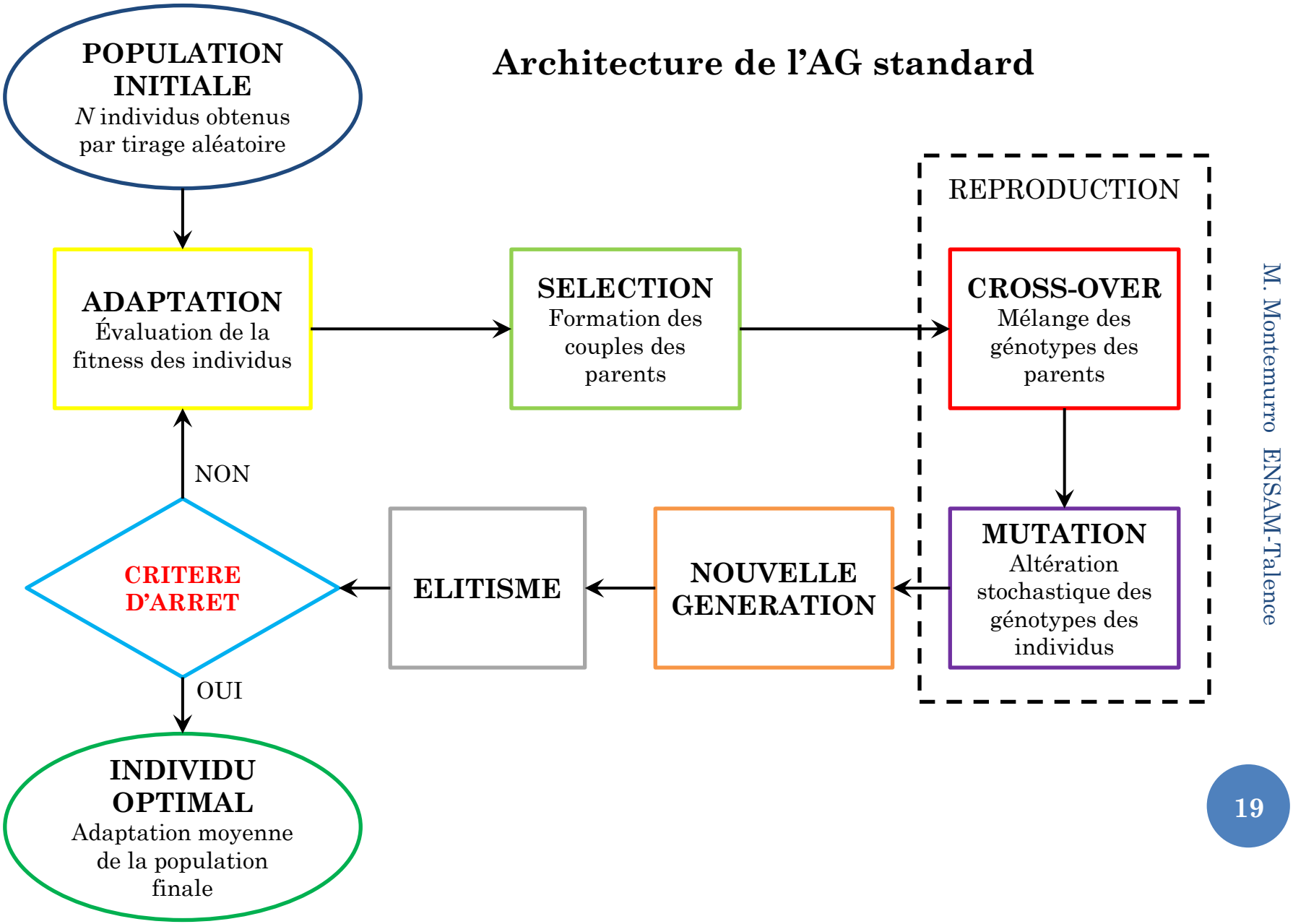
$\boldsymbol{x} \Leftrightarrow \boldsymbol{i}$ Ce qui est codé en binaire sont les composantes du vecteur $\boldsymbol{i}$

$$\lambda = \left\lceil \frac{\ln n_{var}}{\ln 2} \right\rceil$$ → Nombre de bits nécessaires pour coder l'ensemble des valeurs des indices

[ ]: Fonction partie entière par excès

REMARQUE: la variable x est associée à un gène

L'AG Standard



L'AG Standard

L'opérateur d'Adaptation et la fonction de Fitness

- La **fonction de fitness** ***f*** mesure l'**adaptation** d'un individu par rapport au critère choisi (elle évalue la qualité d'un individu en tant que solution potentielle d'un problème donné)

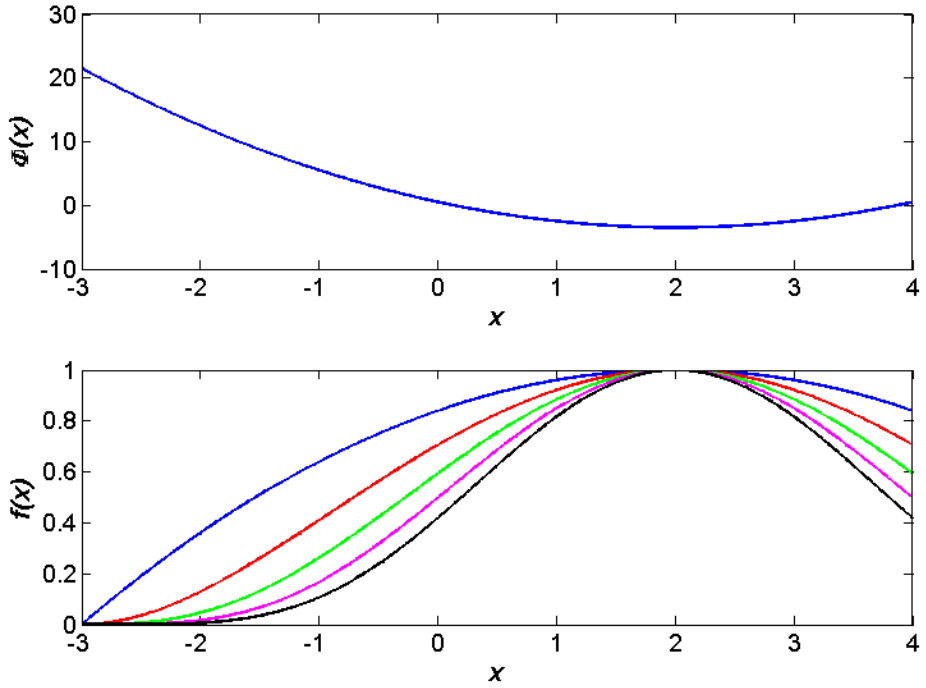
- Le choix de cette fonction n'est pas unique.

Définition

$$f : A \subseteq \mathbb{R} \rightarrow [0, 1]$$

f = 0 *(individu le moins adapté)*

f = 1 *(individu le mieux adapté)*



L'AG Standard

L'opérateur de Sélection

- Sur la base des valeurs de fitness des individus, *N*/2 couples d'individus sont choisis pour la phase de reproduction

- Cette sélection est faite sur la base du concept que les **individus les mieux adaptés** (ceux avec *f* plus grande) ont une probabilité accrue de se reproduire par rapport aux autres

- Parallélisme avec la Nature: les individus plus "forts" (les mieux adaptés) ont une probabilité accrue de vivre plus longtemps par rapport aux autres et donc ils ont plus de temps pour se reproduire

- La **sélection naturelle** s'opère donc sur la base de l'adaptation et de la probabilité de survie

L'AG Standard

L'opérateur de Sélection

Types de sélection plus employés

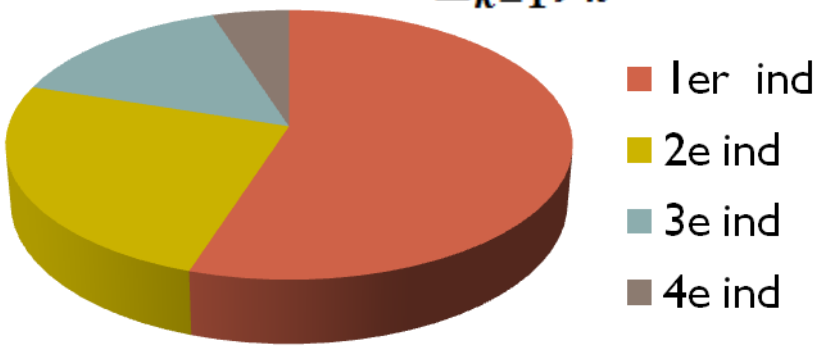
- Tournoi entre ***k*** individus
- Elitisme ***n/2***
- Roue de loterie biaisée

Exemple : sélection par roue de loterie biaisée

Probabilité de sélection du j^{ème} individu

$$(p_{sel})_j = \frac{f_j}{\sum_{k=1}^{N} f_k}$$

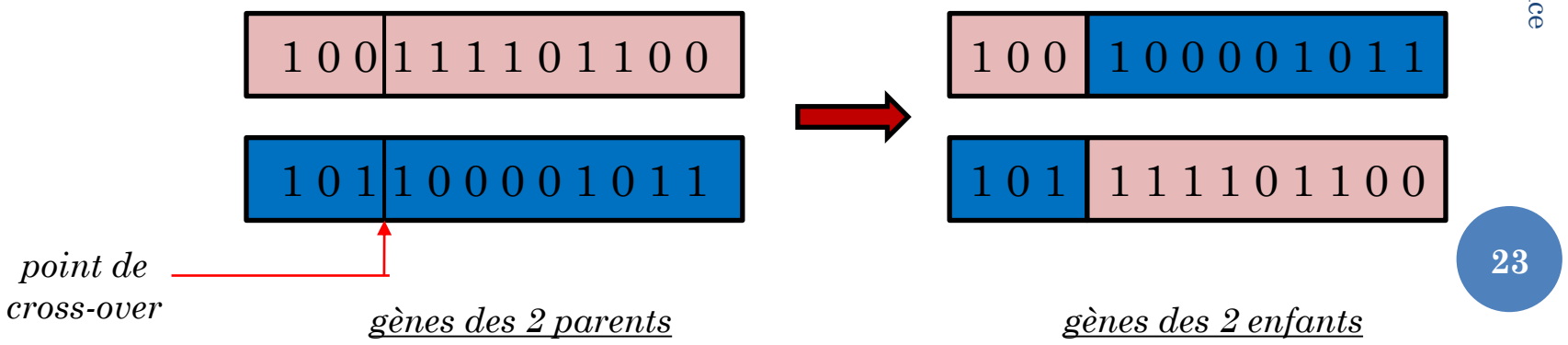
Les ***N/2*** couples sont tirés au hasard par rotation de la roue



L'AG Standard

L'opérateur de Croisement (ou cross-over)

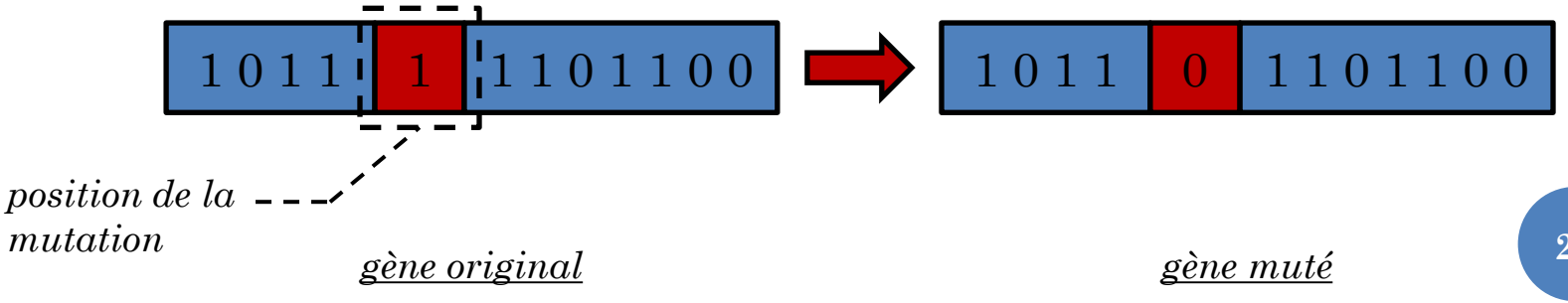
- Le ***croisement*** est l'opérateur principal : le génotype du couple des parents est recombiné pour donner deux nouveaux individus
- Le cross-over a lieu avec une probabilité p_c
- Plusieurs types de croisement sont possibles: en un point, en deux points, continu (ou réel), moyen, etc.
- Cross-over en un point (ou simple): <u>la position unique du cross-over est choisie au hasard</u>



L'AG Standard

L'opérateur de Mutation

- La **mutation** est un **changement aléatoire** d'un des allèles (bits) d'un gène pour un individu
- La mutation a lieu avec une probabilité p_m et sa position est choisie au hasard
- Le rôle de la mutation est celui de "créer de la biodiversité" pour **contraster la convergence prématurée de l'AG** vers une solution pseudo-optimale



L'AG Standard

L' opérateur d' élitisme

Dans chaque génération il existe un meilleur individu

Afin d'éviter de faire disparaitre (au cours des générations) le patrimoine génétique de cet individu on remplace le pire individu de la nouvelle génération avec le meilleur de celle précédente

Le critère d'arrêt

1. Après un nombre fixé de générations $\mathbf{N_{gen}}$
2. Après que la fonction objectif a attendu à une valeur seuil **P**
3. Après que la fonction objectif n'a pas amélioré sa valeur pour un certain nombre de générations

L'AG Standard

Exemple numérique

- Tous les exemples numériques montrés dans ce cours sont abordés et résolus avec l'Algorithme Génétique BIANCA (*Biologically Inspired ANalysis of Composite Assemblages*)

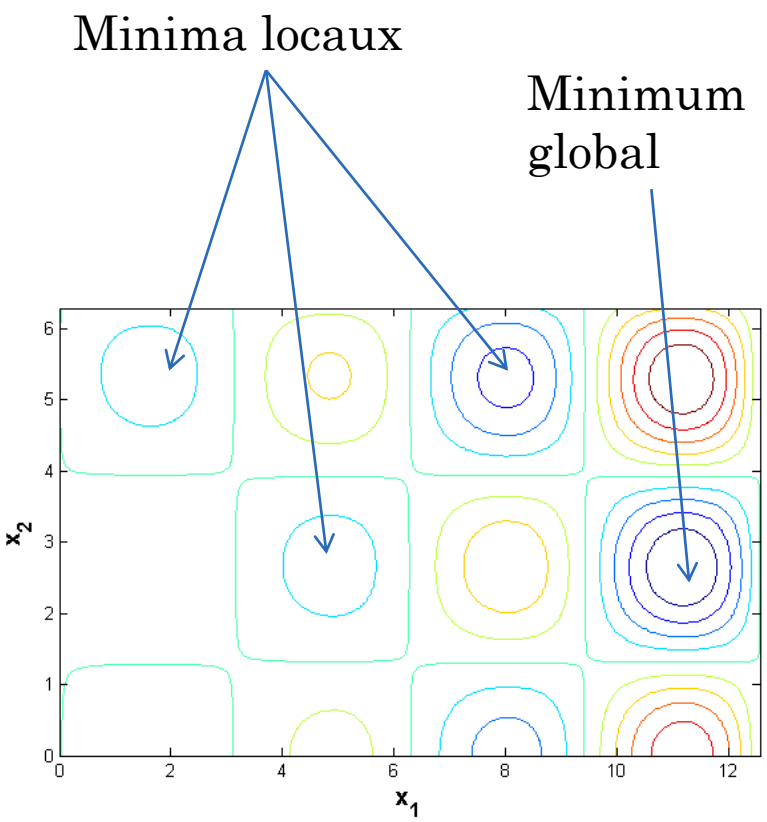
- Renseignements ultérieurs sur l' architecture de la dernière version de BIANCA sont disponibles dans le références suivants
  - M. Montemurro, P. Vannucci & A. Vincenti: BIANCA, a Genetic Algorithm for Engineering Optimisation - User guide. hal-00767468, 2012.
  - M. Montemurro. Optimal Design of Advanced Engineering Modular Systems through a new Genetic Approach. PhD Thesis, Université Pierre et Marie Curie Paris VI, France, 2012.

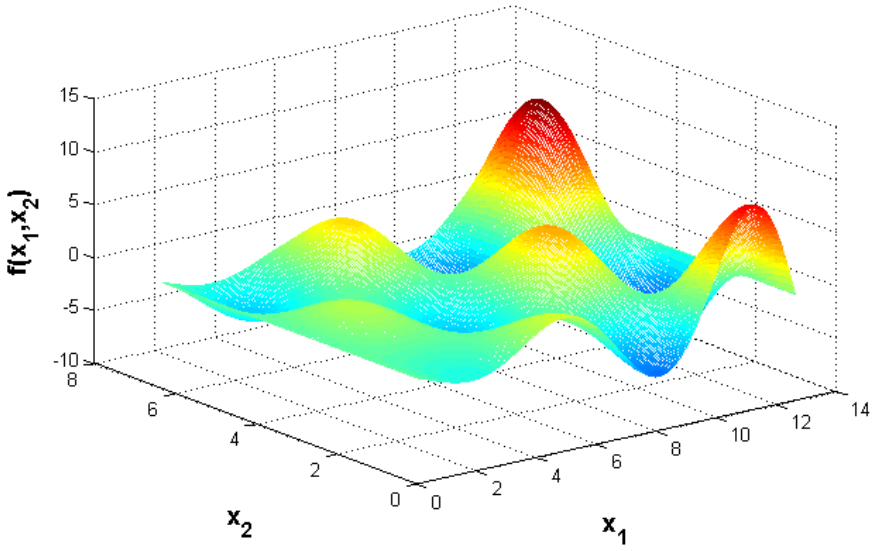
L'AG Standard

Exemple numérique

$$k = 0.2$$
$$a = 1$$
$$b = 0.6$$

$$\begin{cases} \min f(x_1, x_2) = -e^{ka\sqrt{x_1^2+x_2^2}} \sin(ax_1)\cos(2bx_2) \\ 0 \le x_1 \le 4\pi \\ 0 \le x_2 \le 2\pi \end{cases}$$





REMARQUE: NLPP non-convexe sans contraintes

L'AG Standard

Exemple numérique

Données génétiques

- 1 population avec $N_{ind} = 100$ individus
- Critère d'arrêt: nombre fixé de générations, $N_{gen} = 200$
- Probabilité de cross-over: $p_c = 0{,}85$
- Probabilité de mutation: $p_m = {}^{1}/_{N_{ind}}$
- Operateur de sélection: roue de loterie biaisée

Résultats

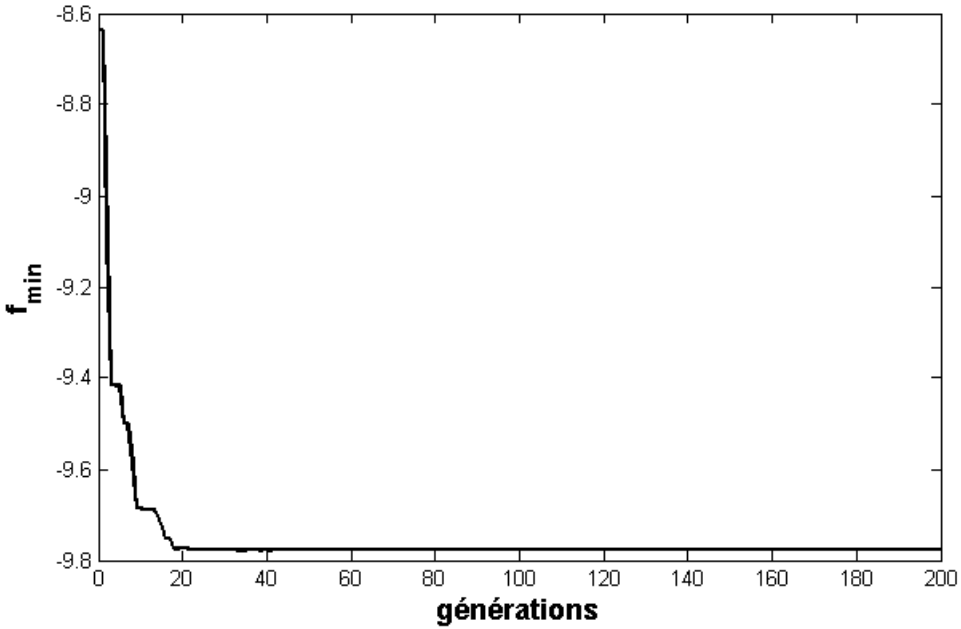
Point optimum: (11,1878; 2,64958)
Valeur optimale FO: -9,77809

Structure de l'individu

1 chromosome avec 2 gènes

x_1	x_2

Finesse de discrétisation:
$\Delta x_1 = 10^{-5}, \Delta x_2 = 10^{-5}$



L'AG Standard

Observations conclusives

1. Méthode de **recherche globale** et non pas locale → bien adaptée pour problèmes non-convexes
2. Parallélisme: l'AG travaille simultanément sur plusieurs points dans l'espace de recherche
3. Codage des variables → l'AG peut manipuler facilement et de façon identique variables de nature différente (continues, discrètes, abstraites)
4. Méthode d'**ordre 0**
5. Les AG sont capables d'accumuler de l'information sur un espace de recherche inconnu (**exploration**) et de l'**exploiter** pour **guider la recherche** sur des sous-espaces prometteurs
6. La recherche se fait par échange d'informations élémentaires (**schèmes ou motifs de similarité**)
7. A chaque génération, l'AG va exploiter un nombre des schèmes proportionnel à N^3 (**théorème du parallélisme implicite)**
8. Les AG sont des algorithmes à formulation libre et, en partie, **auto-correcteurs**

Le traitement des contraintes dans le NLPP

Problème de Programmation Non-Linéaire (NLPP)

$$\min_{\mathbf{x}} \Phi(\mathbf{x})$$

$$\begin{cases} g_i(\mathbf{x}) \le 0 & i = 1,\dots,r \\ h_j(\mathbf{x}) = 0 & j = 1,\dots,m \\ \mathbf{x}_L \le \mathbf{x} \le \mathbf{x}_U & \mathbf{x}_L, \mathbf{x}_U, \mathbf{x} \in \Re^n \end{cases}$$

Il y a essentiellement deux stratégies numériques pour ce type de problèmes

- méthodes des **multiplicateurs**: méthodes duale, du lagrangien augmenté, etc.
- méthodes de **pénalisation et des barrières**: on ne utilise pas les multiplicateurs (et donc les informations données par les conditions d'optimalité)

Le traitement des contraintes dans le NLPP

- En toute généralité (pour toutes les méthodes précédemment introduites), le NLPP avec contraintes est réduit à un **NLPP sans contraintes**

Méthodes de pénalisation: formulation mathématique

NLPP sans contraintes

$$\min_{\mathbf{x}} \Phi_p(\mathbf{x})$$

Φ_p: fonction objectif pénalisée

$$\Phi_p(\mathbf{x}) = \begin{cases} \Phi(\mathbf{x}) & \text{if } g_i(\mathbf{x}) \le 0 \ \wedge \ h_j(\mathbf{x}) = 0 \ , \\ \Phi(\mathbf{x}) + \sum_{i=1}^{r} c_i G_i(\mathbf{x}) + \sum_{j=1}^{m} q_j H_j(\mathbf{x}) & \text{if } g_i(\mathbf{x}) > 0 \ \wedge \ h_j(\mathbf{x}) \neq 0 \ , \end{cases}$$

$$i = 1, ..., r \ , \ j = 1, ..., m \ ,$$

$$G_i(\mathbf{x}) = \max[0, g_i(\mathbf{x})] \qquad i = 1, ..., r \ ,$$

$$H_j(\mathbf{x}) = \max[0, |h_j(\mathbf{x})| - \epsilon] \qquad j = 1, ..., m \ .$$

c_i: coefficients de pénalisation des contraintes d'inégalité
q_j: coefficients de pénalisation des contraintes d' égalité

REMARQUE: les contraintes d'égalité $h_j = 0$ sont transformées en contraintes d'inégalité équivalentes du type $|h_j| - \epsilon \le 0$

Le traitement des contraintes dans le NLPP

Méthodes de pénalisation

- L'idée de ces méthodes est de transformer le NLPP avec contraintes de telle sorte que les solutions non-faisables (ou non admissibles, c'est-à-dire qui ne satisfont pas au moins une contrainte) soient pénalisées selon un critère donné

- Comment choisir le critère de pénalisation? Ou, de manière équivalente, comment choisir des valeurs appropriées pour les coefficients c_i et q_j?

Différents types de méthodes de pénalisation:

1. Pénalisation Statique

2. Pénalisation Dynamique

3. Pénalisation Adaptative et Dynamique: la méthode ADP

M. Montemurro, A. Vincenti & P. Vannucci: The Automatic Dynamic Penalisation method (ADP) for handling constraints with genetic algorithms. *Computer Methods in Applied Mechanics and Engineering*, v. 256, pp. 70-87, 2013.

Le traitement des contraintes dans le NLPP

Pénalisation Statique

$$\Phi_p(\boldsymbol{x}) = \Phi(\boldsymbol{x}) + c\sum_{i=1}^{r}\left(G_i\ (\boldsymbol{x})\right)^{\alpha} + q\sum_{j=1}^{m}\left(H_j\ (\boldsymbol{x})\right)^{\beta} \qquad \alpha, \beta > 1$$

Défauts:

- c et q sont les coefficients de pénalisation qui doivent être définis par l'utilisateur
- problème → bien cibler la valeur de chacun des coefficients!!!
- les valeurs de c et q dépendent beaucoup de la nature du NLPP et souvent il faut faire des tests numériques afin de choisir des valeurs appropriées
- un seul coefficient pour toutes les contraintes de même nature (égalité et inégalité) du NLPP

Le traitement des contraintes dans le NLPP

Pénalisation Dynamique

$$\Phi_p(\boldsymbol{x}) = \Phi(\boldsymbol{x}) + c(t)\sum_{i=1}^{r}\left(G_i\ (\boldsymbol{x})\right)^{\alpha} + q(t)\sum_{j=1}^{m}\left(H_j\ (\boldsymbol{x})\right)^{\beta} \qquad \alpha, \beta > 1$$

t : itération courante

$c(t) = c_0\psi(t)$
$q(t) = q_0\xi(t)$

$\psi(t)$ et $\xi(t)$ sont des fonctions arbitraires du nombre d'itérations

- L'idée à la base des méthodes de pénalisation dynamiques est que les coefficients de pénalisation sont mis à jour pendant l'évolution de la population (au cours des itérations)
- En pratique, les fonctions $\psi(t)$ et $\xi(t)$ sont choisies de façon telle que les coefficients $c(t)$ et $q(t)$ prennent des valeurs petites pendant les premières générations (afin de permettre à l'AG d'explorer tout l'espace) et des valeurs plus importants pendant les itérations finales

Défauts:

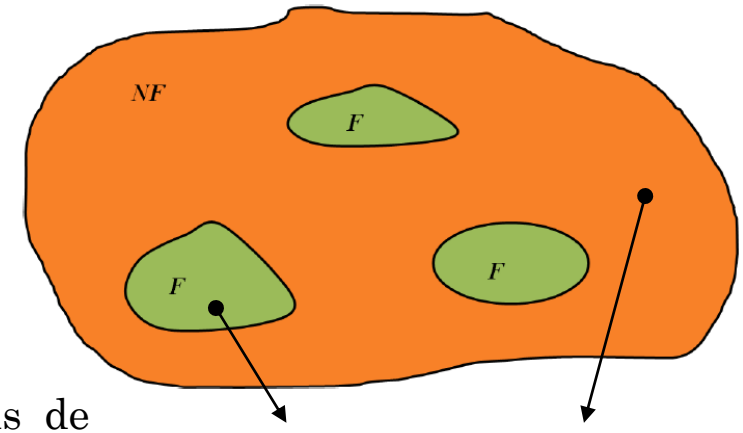
En pratique, les mêmes des méthodes statiques car c'est l'utilisateur qui doit définir correctement les valeurs de c_0 et q_0 ainsi que des paramètres auxiliaires éventuels qui interviennent dans la définition des fonctions $\psi(t)$ et $\xi(t)$

Le traitement des contraintes dans le NLPP

Pénalisation Adaptative et Dynamique: méthode ADP (*Automatic Dynamic Penalisation*)

$$c_i(t) = \frac{|\Phi^F_{best} - \Phi^{NF}_{best}|}{(G_i)^{NF}_{best}} \quad i = 1, ..., r \ ,$$

$$q_j(t) = \frac{|\Phi^F_{best} - \Phi^{NF}_{best}|}{(H_j)^{NF}_{best}} \quad j = 1, ..., m \ .$$



F: région faisable

NF: région non-faisable

- Les coefficients de pénalisation sont choisis de façon automatique (sans l'intervention de l'utilisateur) et ils sont mis à jour de façon dynamique à chaque itération par l'AG
- La valeur de chaque coefficient est évaluée de manière adaptative par l'AG lui-même sur la base de la distribution des individus (faisables et non-faisables) à l'itération courant
- Chaque coefficient dépend de l'information génétique (en termes de fonction objectif et contraintes) du meilleur individu de la région *F* ($\Phi_{best}{}^{F}$) et de celle du meilleur de la région *NF* ($\Phi_{best}{}^{NF}, (G_i)_{best}{}^{NF}, (H_j)_{best}{}^{NF}$)
- L'idée principale à la base de l'ADP est que certaines solutions *NF* très performantes (en termes de fonction objectif) sont retenues dans la population et agissent en tant que points d'attraction pour l'AG en améliorant ses propriétés de recherche surtout à la frontière entre les régions *F* et *NF*

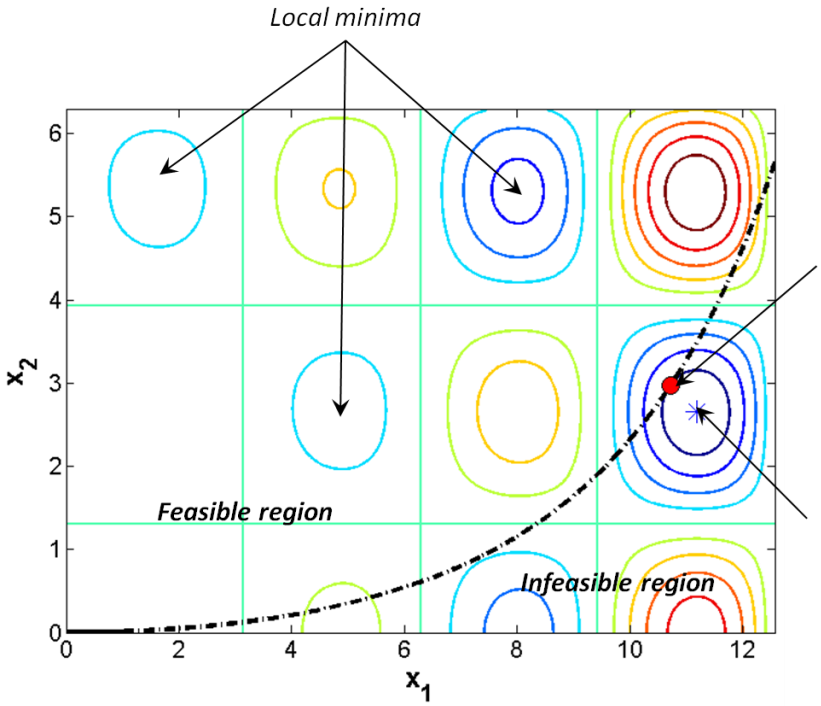
La méthode ADP: exemple numérique

$$\min f(x_1, x_2) = -e^{ka\sqrt{x_1^2+x_2^2}} \sin(ax_1)\cos(2bx_2)$$

$k = 0.2$
$a = 1$
$b = 0.6$
$c = 0.012$

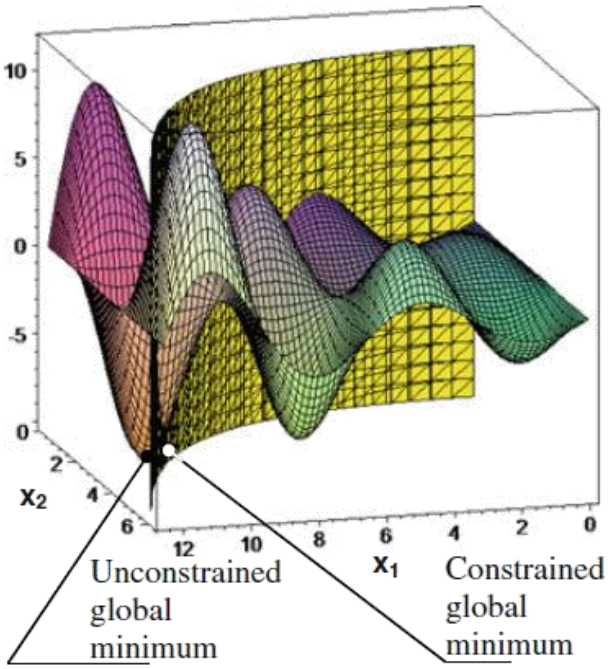
subject to:

$$x_2 > e^{cx_1^2} - 1$$
$$0 \le x_1 \le 4\pi$$
$$0 \le x_2 \le 2\pi$$



Constrained global minimum

Unconstrained global minimum



REMARQUE: NLPP non-convexe avec contrainte non-linéaire

La méthode ADP: exemple numérique

Données génétiques

- 1 population avec $N_{ind} = 100$ individus
- Critère d'arrêt: nombre fixé de générations, $N_{gen} = 200$
- Probabilité de cross-over: $p_c = 0{,}85$
- Probabilité de mutation: $p_m = {}^{1}/_{N_{ind}}$
- Operateur de sélection: roue de loterie biaisée

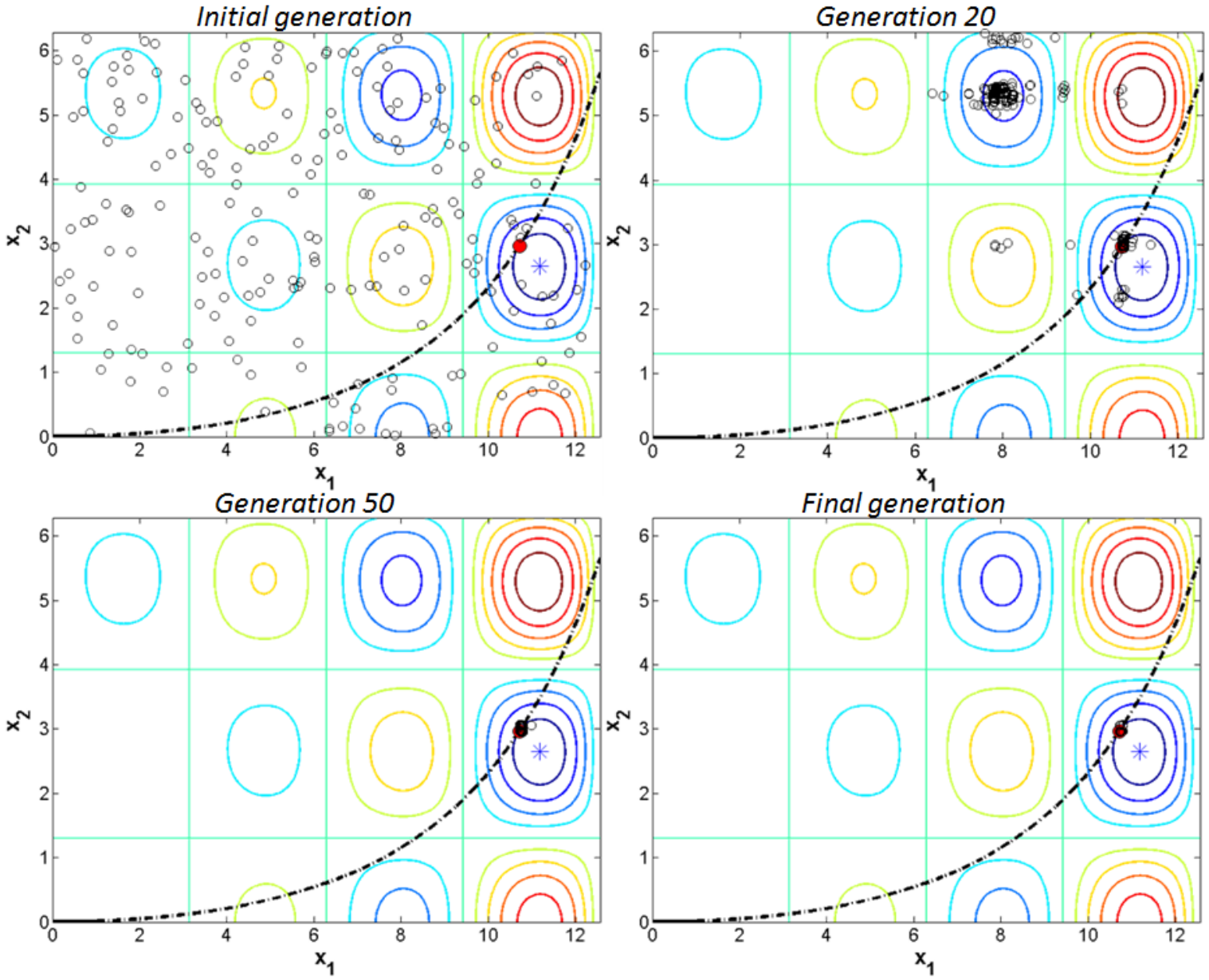
Structure de l'individu

x_1	x_2

1 chromosome avec 2 gènes

Finesse de discrétisation:
$\Delta x_1 = 10^{-5}, \Delta x_2 = 10^{-5}$

La méthode ADP: exemple numérique

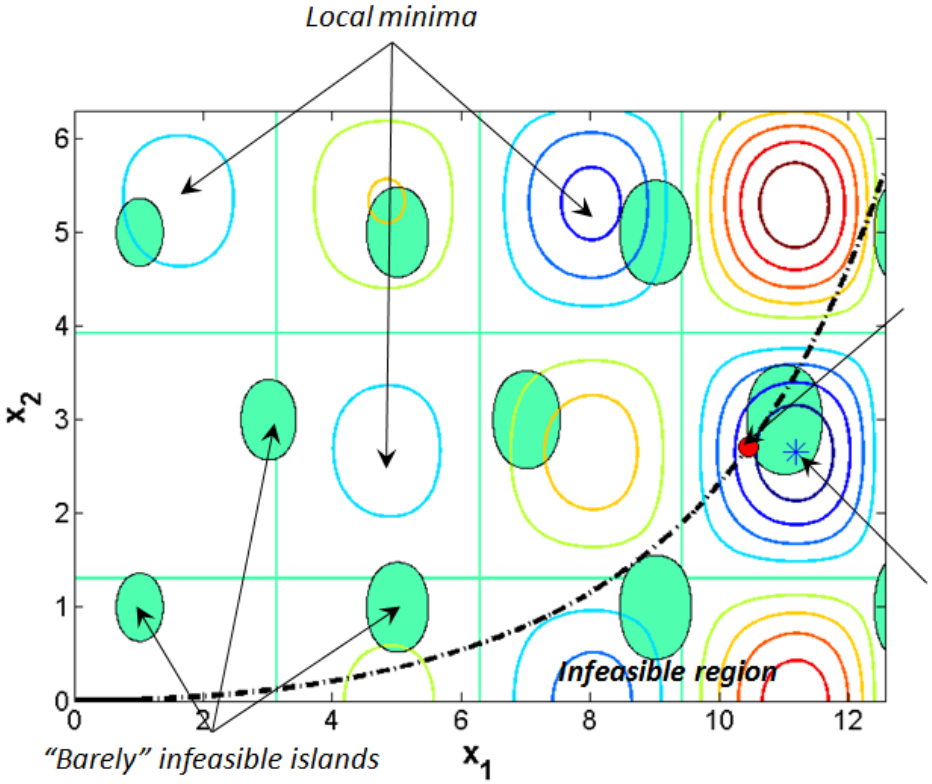


La méthode ADP: exemple numérique (2)

$$\min_{\mathbf{x}} \Phi(x_1,x_2) = -\mathrm{e}^{ka\sqrt{x_1^2+x_2^2}} \sin(ax_1)\cos(2bx_2)$$

subject to :

$$\begin{cases} g_1(x_1,x_2) = \mathrm{e}^{cx_1^2} - 1 - x_2 \le 0 \\ g_2(x_1,x_2) = 0.1\left[\left(\dfrac{0.4}{4\pi}x_1 + 0.8\right)\sin\left(\dfrac{\pi}{2}x_1\right)\sin\left(\dfrac{\pi}{2}x_2\right) - 0.7\right] \le 0 \\ 0 \le x_1 \le 4\pi \quad 0 \le x_2 \le 2\pi \end{cases}$$



REMARQUE: NLPP non-convexe avec contrainte non-linéaire défini sur un domaine disjoint (2ème contrainte non-convexe)

La méthode ADP: exemple numérique (2)

Données génétiques

- 1 population avec $N_{ind} = 150$ individus
- Critère d'arrêt: nombre fixé de générations, $N_{gen} = 200$
- Probabilité de cross-over: $p_c = 0{,}85$
- Probabilité de mutation: $p_m = {}^{1}/_{N_{ind}}$
- Operateur de sélection: roue de loterie biaisée

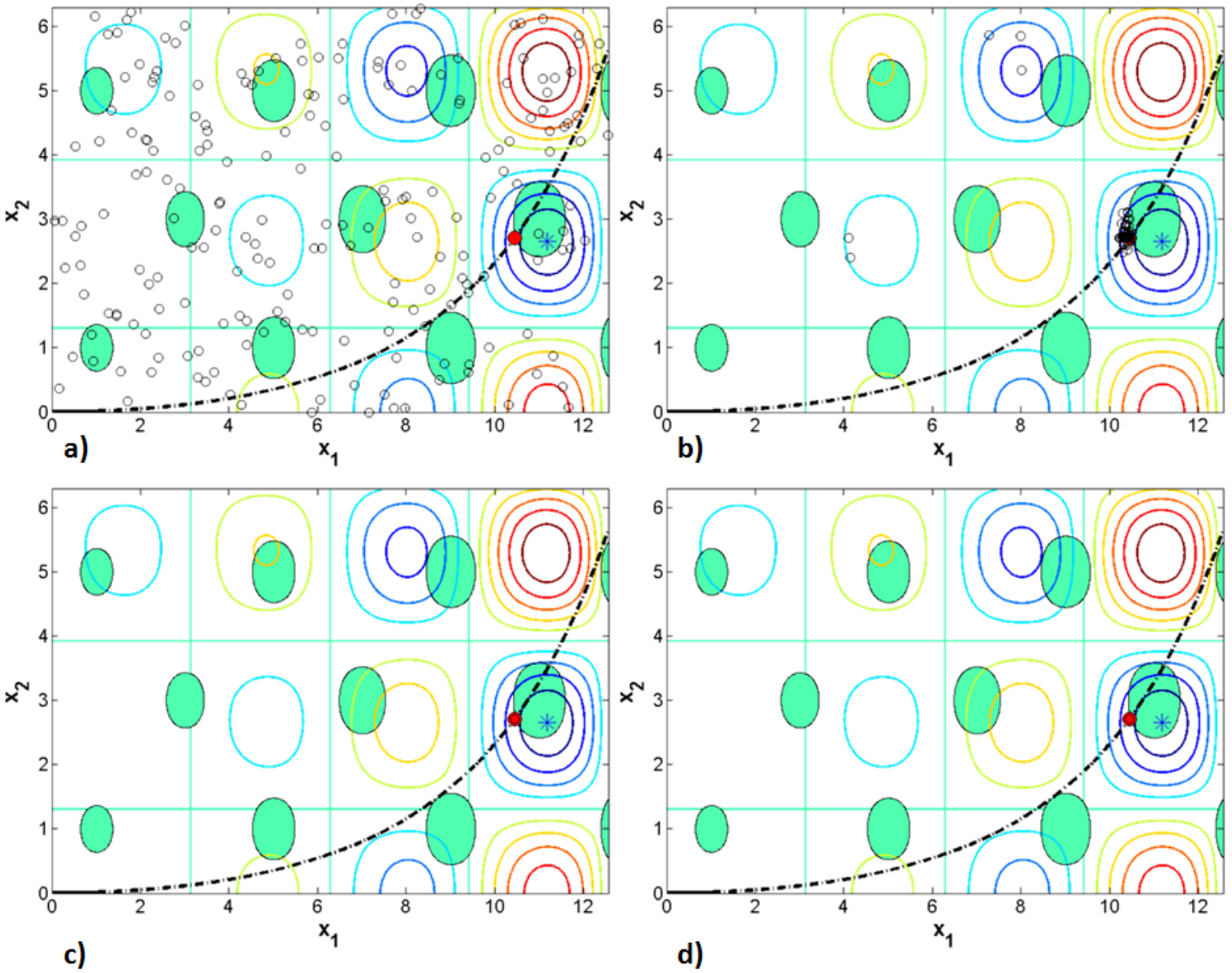
Structure de l'individu

x_1	x_2

1 chromosome avec 2 gènes

Finesse de discrétisation:
$\Delta x_1 = 10^{-5}, \Delta x_2 = 10^{-5}$

La méthode ADP: exemple numérique (2)

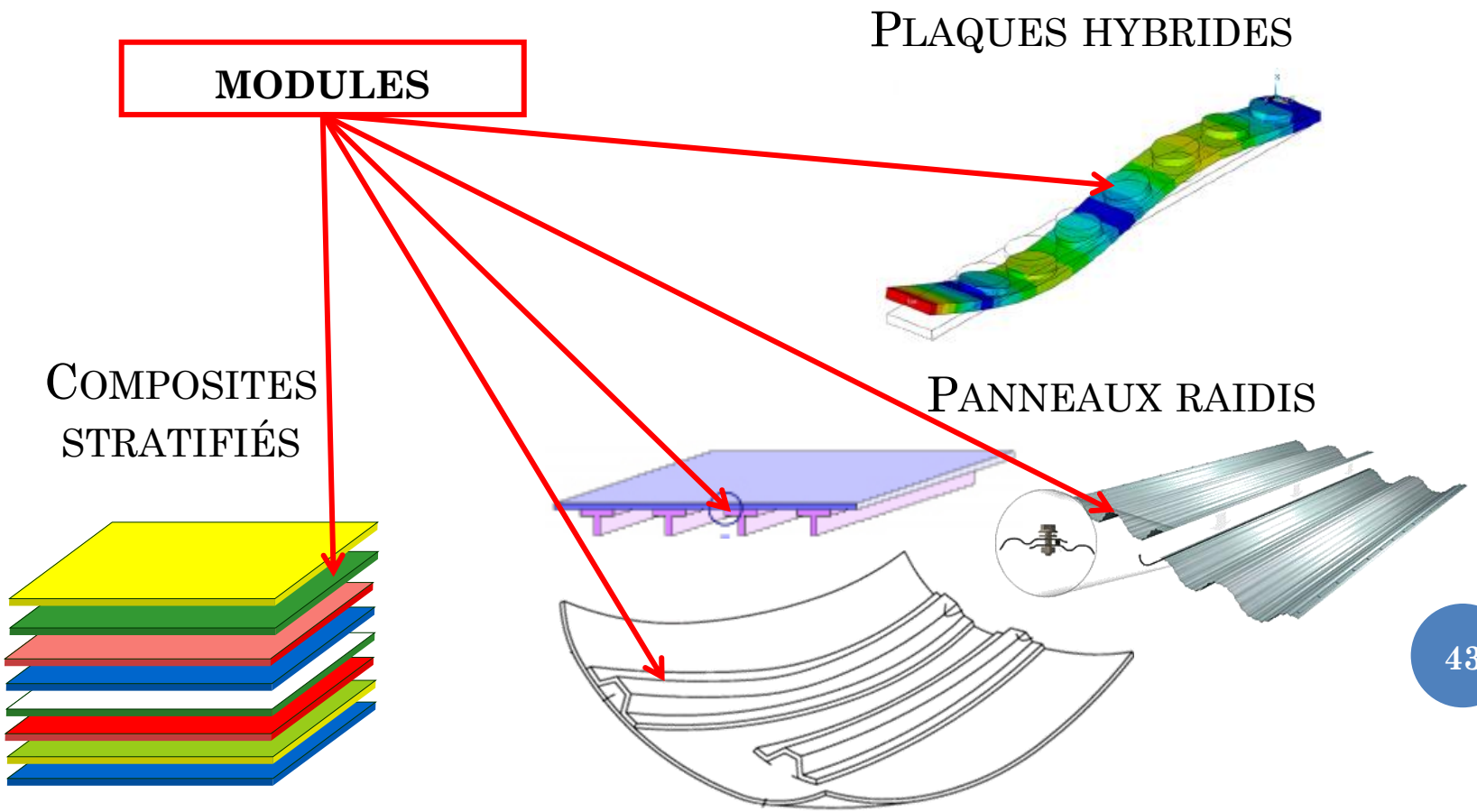


L'Optimisation des Systèmes Modulaires

L'optimisation des systèmes modulaires: problématique

- **Système modulaire** → système composé par « unités élémentaires », notamment les *modules*

- **Module** → chaque module est caractérisé par le même vecteur d'inconnues, c'est-à-dire les *variables de conception* du module



L'optimisation des systèmes modulaires: formulation mathématique

$$\min_{\mathbf{x}} \Phi(\mathbf{x})$$

subject to:

$$\begin{cases} g_i(\mathbf{x}) \le 0 & i = 1,\ldots,r(N) \\ h_j(\mathbf{x}) = 0 & j = 1,\ldots,m(N) \\ \mathbf{x}_L \le \mathbf{x} \le \mathbf{x}_U & \mathbf{x}_L, \mathbf{x}_U, \mathbf{x} \in \Re^{N_{\text{var}}(N)} \end{cases}$$

N **Nombre de modules**

$N_{\text{var}}(N)$ **Nombre de variables de conception**

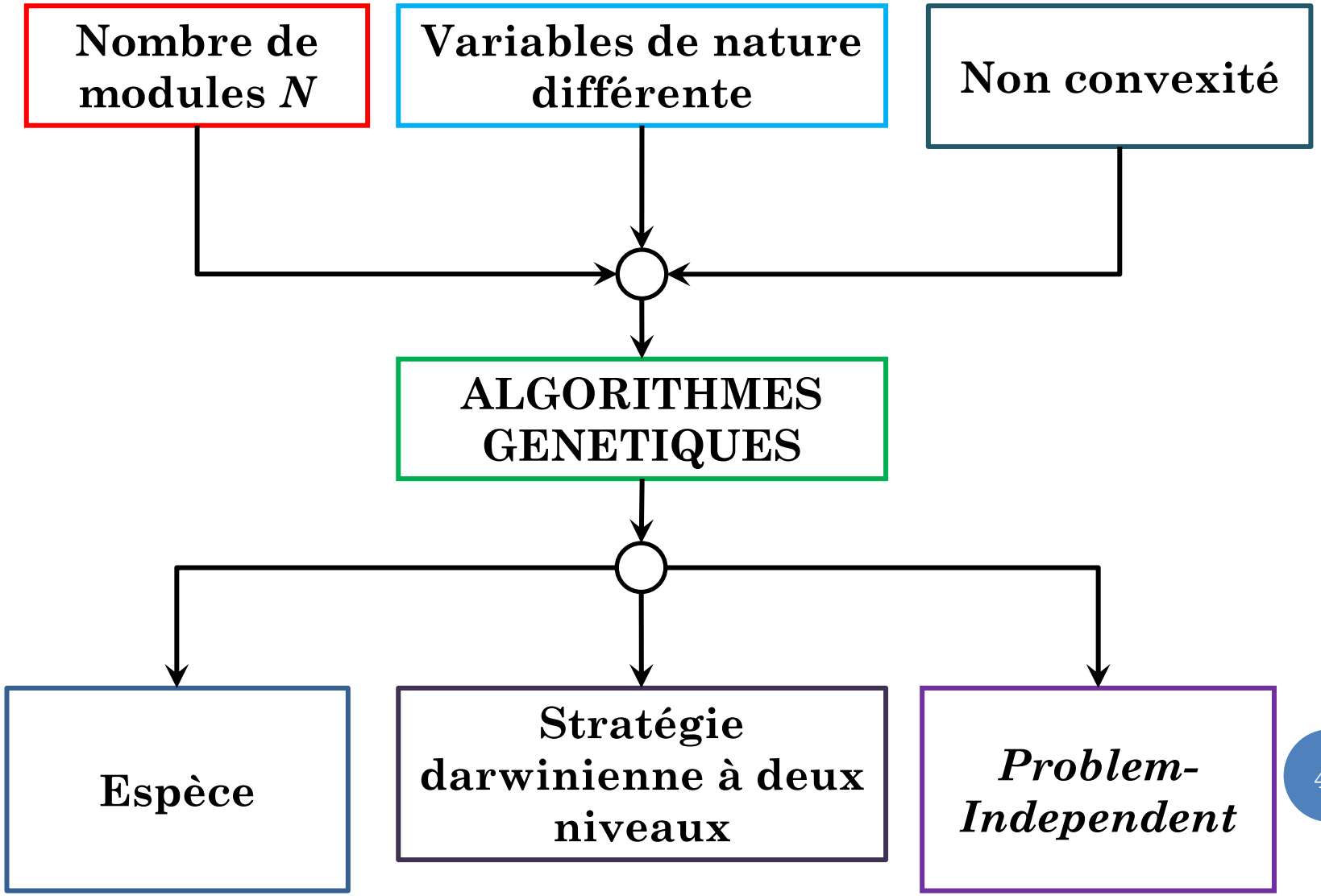
Espace de conception à dimension variable

L'optimisation des systèmes modulaires: formulation mathématique

Difficultés:

- comment aborder un problème d'optimisation défini sur un espace à dimension variable?
- comment inclure le nombre de modules (et donc la dimension de l'espace) parmi les variables de conception du problème?
- optimisation simultanée du nombre de modules ainsi que des paramètres caractéristiques de chaque module (les modules sont, *a priori*, tous différents)
- variables de nature différente (continues, discrètes, abstraites…)
- non-convexité du problème

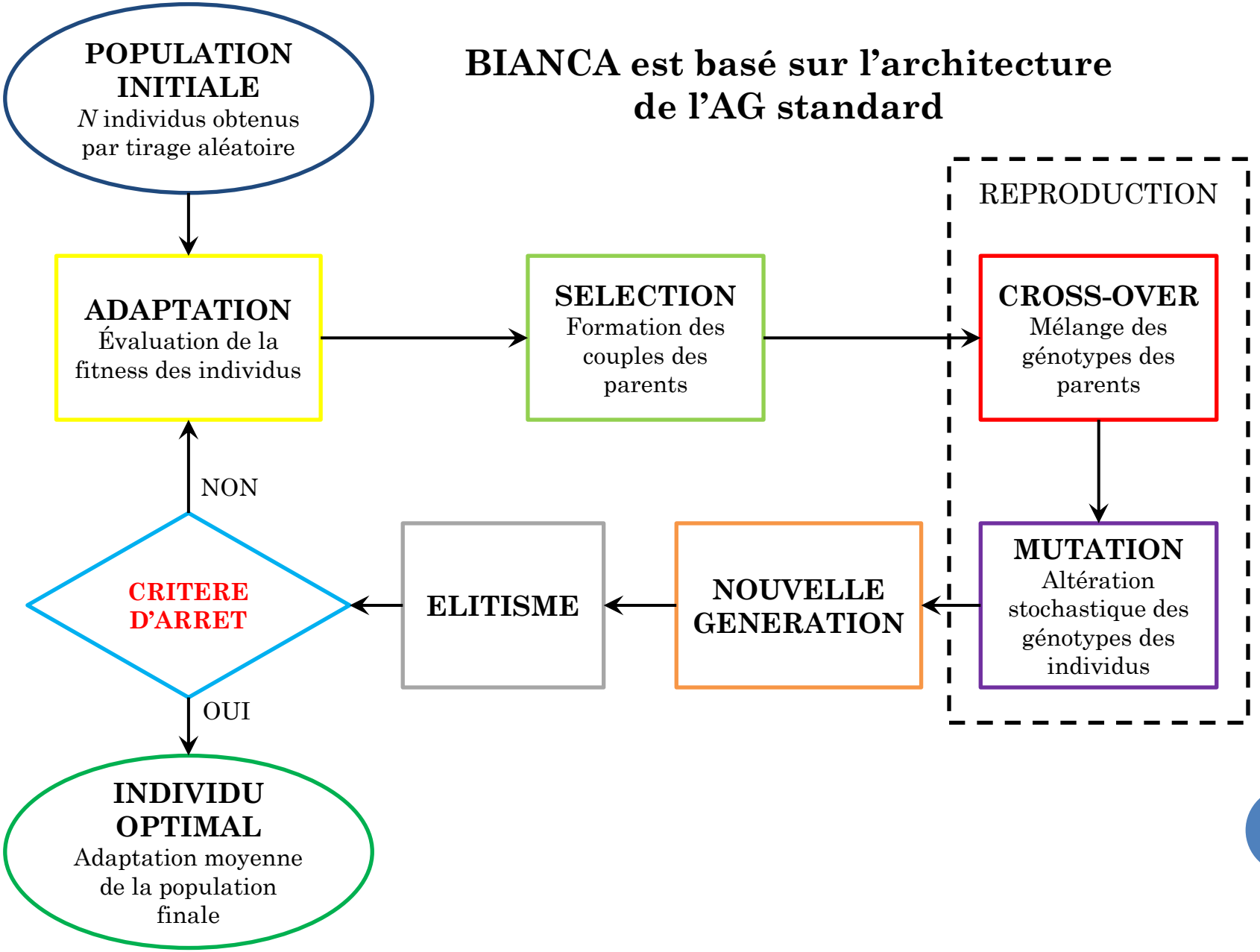
L'optimisation des systèmes modulaires: approche proposée



L'Algorithme Génétique BIANCA

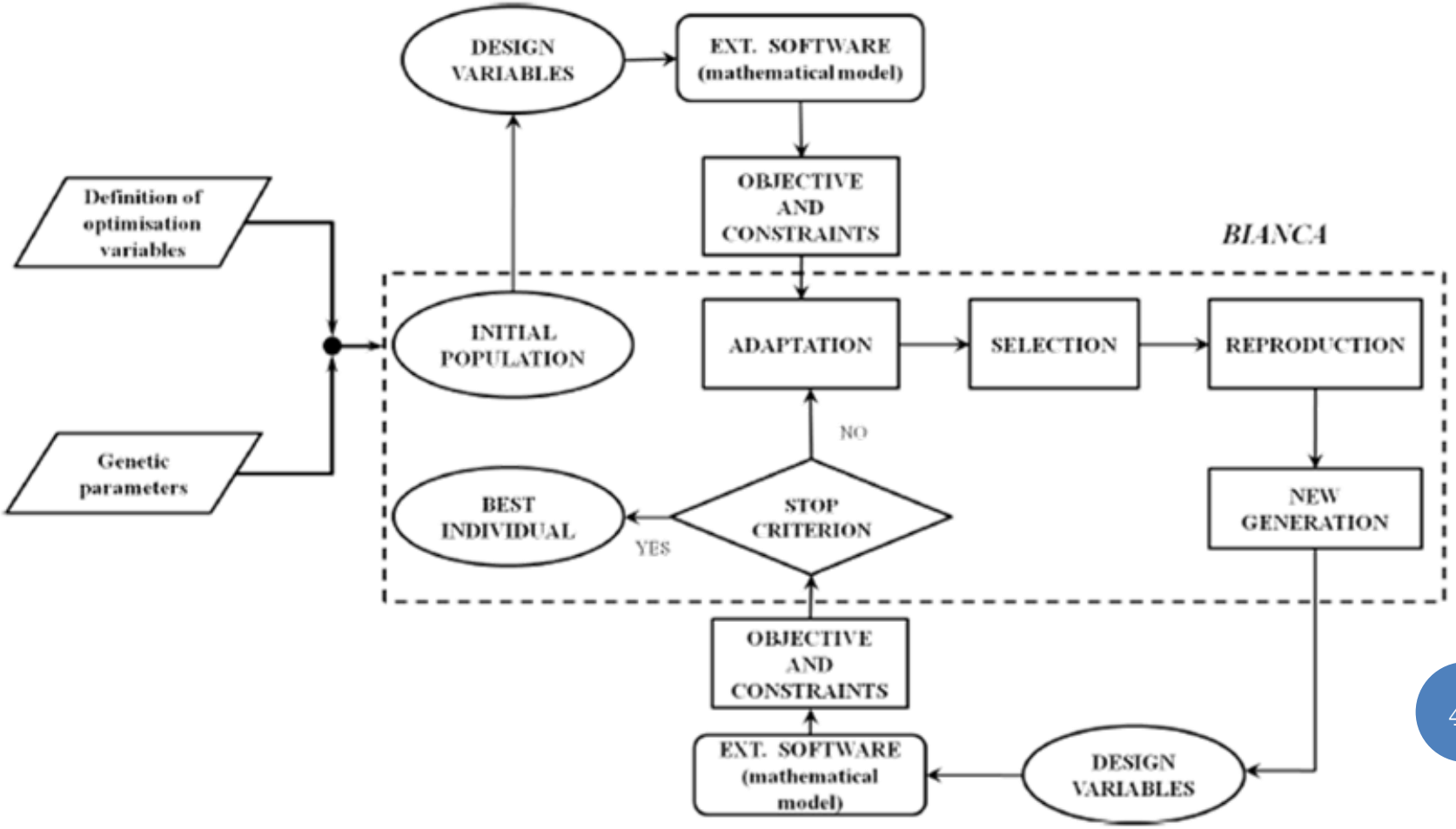
(Biologically Inspired ANalysis of Composite Assemblages)

L'AG BIANCA


POPULATION INITIALE
N individus obtenus par tirage aléatoire
BIANCA est basé sur l'architecture de l'AG standard
REPRODUCTION
ADAPTATION
Évaluation de la fitness des individus
SELECTION
Formation des couples des parents
CROSS-OVER
Mélange des génotypes des parents
MUTATION
Altération stochastique des génotypes des individus
NOUVELLE GENERATION
ELITISME
CRITERE D'ARRET
NON
OUI
INDIVIDU OPTIMAL
Adaptation moyenne de la population finale

L'AG BIANCA: fonctionnalités supplémentaires

- ✓ **Croisement et mutation entre espèces différentes**
- ✓ **Méthode ADP** pour la prise en compte des contraintes
- ✓ **Interface avec codes externes**
- ✓ **Interface graphique utilisateur (GUI)**



L'AG BIANCA: représentation des informations

Individu : *point solution dans l'espace de conception* (p.ex. stratifié)

Chromosome : *module constitutif* (p.ex. couche)

Gène : *variable de conception caractérisant chaque module* (p.ex. orientation de la couche)

Espèce : groupe d'individus ayant le même nombre de chromosomes (p.ex. stratifiés avec le même nombre de couches)



NOTE : dans BIANCA les opérateurs génétiques agissent sur chaque gène

L'AG BIANCA: croisement entre espèces

1. Couple de parents

$(\alpha_1)^{P1}$	$(\beta_1)^{P1}$	3
$(\alpha_2)^{P1}$	$(\beta_2)^{P1}$	
$(\alpha_3)^{P1}$	$(\beta_3)^{P1}$	
e	e	
e	e	
e	e	

$(\alpha_1)^{P2}$	$(\beta_1)^{P2}$	5
$(\alpha_2)^{P2}$	$(\beta_2)^{P2}$	
$(\alpha_3)^{P2}$	$(\beta_3)^{P2}$	
$(\alpha_4)^{P2}$	$(\beta_4)^{P2}$	
$(\alpha_5)^{P2}$	$(\beta_5)^{P2}$	
e	e	

2. Opérateur "*SHIFT*"

e	e	3
$(\alpha_1)^{P1}$	$(\beta_1)^{P1}$	
$(\alpha_2)^{P1}$	$(\beta_2)^{P1}$	
$(\alpha_3)^{P1}$	$(\beta_3)^{P1}$	
e	e	
e	e	

$(\alpha_1)^{P2}$	$(\beta_1)^{P2}$	5
$(\alpha_2)^{P2}$	$(\beta_2)^{P2}$	
$(\alpha_3)^{P2}$	$(\beta_3)^{P2}$	
$(\alpha_4)^{P2}$	$(\beta_4)^{P2}$	
$(\alpha_5)^{P2}$	$(\beta_5)^{P2}$	
e	e	

3. Croisement sur chaque gène

e	e	3
$(\alpha_1)^{P1}$	$(\beta_1)^{P1}$	
$(\alpha_2)^{P1}$	$(\beta_2)^{P1}$	
$(\alpha_3)^{P1}$	$(\beta_3)^{P1}$	
e	e	
e	e	

$(\alpha_1)^{P2}$	$(\beta_1)^{P2}$	5
$(\alpha_2)^{P2}$	$(\beta_2)^{P2}$	
$(\alpha_3)^{P2}$	$(\beta_3)^{P2}$	
$(\alpha_4)^{P2}$	$(\beta_4)^{P2}$	
$(\alpha_5)^{P2}$	$(\beta_5)^{P2}$	
e	e	

P1 = *parent* 1
P2 = *parent* 2
e = *empty location*

Observations

- Le croisement est fait sur chaque gène homologue du génotype
- Grâce à l'opérateur de *shift* plusieurs combinaisons de croisements sont possibles

L'AG BIANCA: croisement entre espèces

4. Individus enfants

e	e	3
$(\alpha_1)^{C1}$	$(\beta_1)^{C1}$	
$(\alpha_2)^{C1}$	$(\beta_2)^{C1}$	
$(\alpha_3)^{C1}$	$(\beta_3)^{C1}$	
e	e	
e	e	

$(\alpha_1)^{C2}$	$(\beta_1)^{C2}$	5
$(\alpha_2)^{C2}$	$(\beta_2)^{C2}$	
$(\alpha_3)^{C2}$	$(\beta_3)^{C2}$	
$(\alpha_4)^{C2}$	$(\beta_4)^{C2}$	
$(\alpha_5)^{C2}$	$(\beta_5)^{C2}$	
e	e	

5. Opérateur "*REORDER*"

$(\alpha_1)^{C1}$	$(\beta_1)^{C1}$	3
$(\alpha_2)^{C1}$	$(\beta_2)^{C1}$	
$(\alpha_3)^{C1}$	$(\beta_3)^{C1}$	
e	e	
e	e	
e	e	

$(\alpha_1)^{C2}$	$(\beta_1)^{C2}$	5
$(\alpha_2)^{C2}$	$(\beta_2)^{C2}$	
$(\alpha_3)^{C2}$	$(\beta_3)^{C2}$	
$(\alpha_4)^{C2}$	$(\beta_4)^{C2}$	
$(\alpha_5)^{C2}$	$(\beta_5)^{C2}$	
e	e	

C1 = *child* 1
C2 = *child* 2
e = *empty location*

Observations

- À la fin de l'étape de *cross-over* entre individus appartenant à des espèces différentes on obtient deux nouveaux individus qui appartiennent toujours aux espèces des parents
- Pour avoir l'évolution de l'espèce (donc, du nombre de chromosomes) il faut considérer l'étape de mutation
- L'opérateur de *shift* agit avec une probabilité p_{shift} (en général ≅ 0,5)

L'AG BIANCA: mutation des espèces

6. Mutation du nombre de chromosomes et opérateurs "*ADDITION / DELETION*"

$(\alpha_1)^{C1}$	$(\beta_1)^{C1}$	2
$(\alpha_2)^{C1}$	$(\beta_2)^{C1}$	
e	e	
e	e	
e	e	
e	e	

$(\alpha_1)^{C2}$	$(\beta_1)^{C2}$	6
$(\alpha_a)^{C2}$	$(\beta_a)^{C2}$	
$(\alpha_2)^{C2}$	$(\beta_2)^{C2}$	
$(\alpha_3)^{C2}$	$(\beta_3)^{C2}$	
$(\alpha_4)^{C2}$	$(\beta_4)^{C2}$	
$(\alpha_5)^{C2}$	$(\beta_5)^{C2}$	

7. Mutation aléatoire de chaque gène

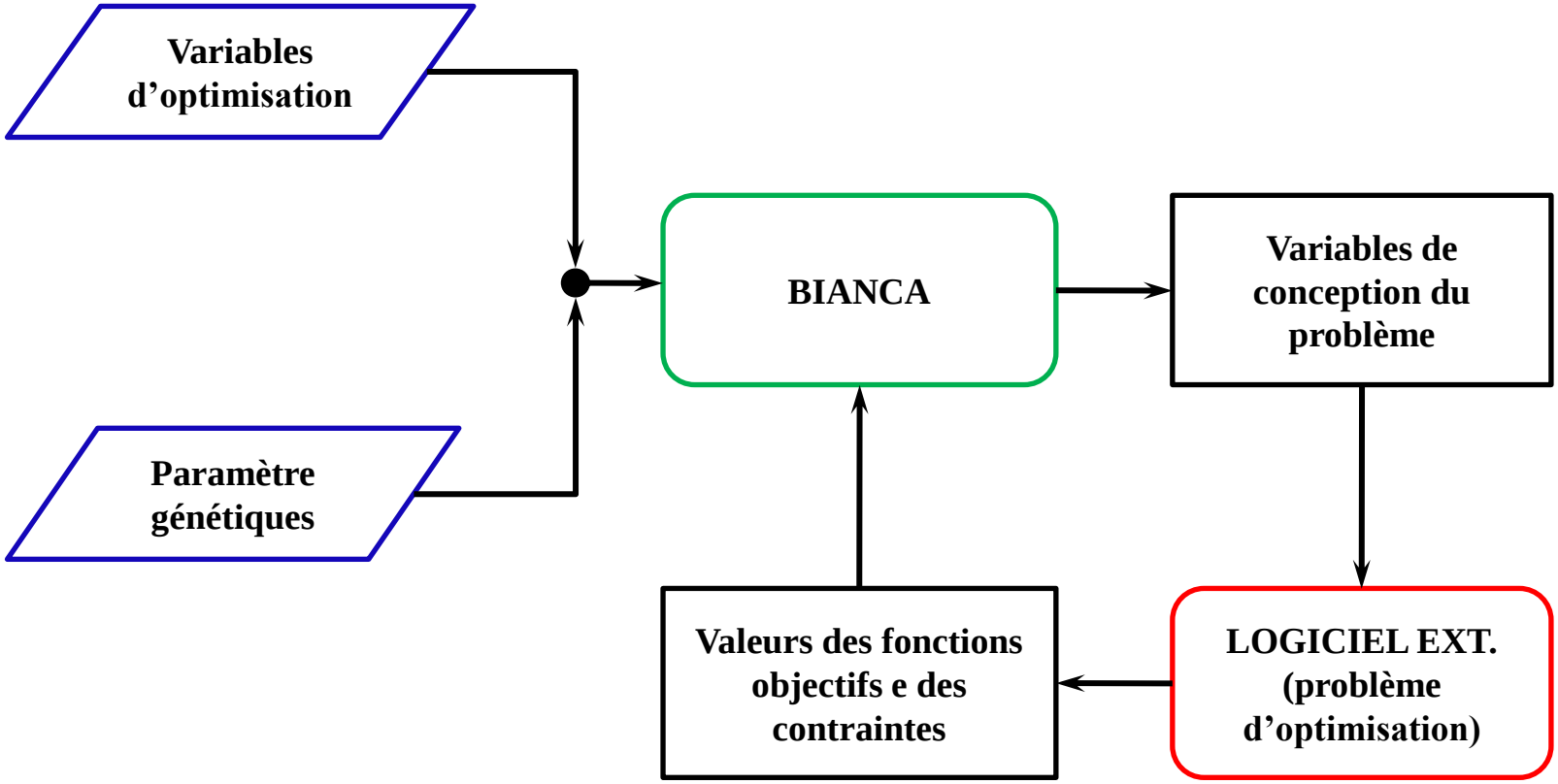
$(\alpha_1)^{C1}$	$(\beta_1)^{C1}$	2
$(\alpha_{2m})^{C1}$	$(\beta_2)^{C1}$	
e	e	
e	e	
e	e	
e	e	

$(\alpha_{1m})^{C2}$	$(\beta_1)^{C2}$	6
$(\alpha_a)^{C2}$	$(\beta_a)^{C2}$	
$(\alpha_2)^{C2}$	$(\beta_2)^{C2}$	
$(\alpha_3)^{C2}$	$(\beta_{3m})^{C2}$	
$(\alpha_4)^{C2}$	$(\beta_4)^{C2}$	
$(\alpha_5)^{C2}$	$(\beta_5)^{C2}$	

Observations

- La mutation du nombre de chromosomes a lieu avec une probabilité $p_{mc} = \frac{(n_{chrom})_{max} - (n_{chrom})_{min}}{N_{ind}}$
- La mutation des gènes a lieu avec une probabilité $p_m = {}^1/_{N_{ind}}$
- Grâce aux nouveaux opérateurs génétiques pour la reproduction (croisement et mutation) entre espèces différentes on est capable d'aborder et résoudre les problèmes d'optimisation des systèmes modulaires, tout simplement en agissant sur le nombre de chromosome du couple

L'AG BIANCA: interface avec logiciels



Logiciels : MATLAB, ABAQUS, ANSYS, NASTRAN-PATRAN

L'AG BIANCA: interface graphique utilisateur

La GUI est réalisée avec le logiciel **MATLAB**

